

**Laxmi Charitable Trust's**  
**Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce**  
**Department of Information Technology (B.Sc.I.T Semester V)**  
**APPLIED BUSINESS ANALYTICS**  
**Practical –VII**

|                               |                                    |
|-------------------------------|------------------------------------|
| Roll No.:T006                 | Name:Nandini Chaudhari             |
| Class:TYIT                    | Batch:01                           |
| Date of Assignment:03-10-2026 | Date/Time of Submission:03-10-2026 |

### Practical 7: Sampling and Sampling Distributions

#### Part A: Survey and Sampling

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
# Load dataset
df = pd.read_csv(
    "Student Welfare Hub Feedback Form (Responses) - Form Responses 1.csv"
)
print("Dataset loaded successfully!")
print("Shape of dataset:", df.shape)
```

... Dataset loaded successfully!  
Shape of dataset: (55, 11)

#### 1. Select a real-life business topic for conducting a survey.

**Answer:-**

Survey Topic: User Feedback Analysis of Student Welfare Hub

#### 2. Prepare a questionnaire containing at least 5 questions related to the selected topic.

**Answer:-**

Number of questionnaire questions: 7

- 1 . What is your current level of study?
- 2 . How easy was it to navigate and use the Student Welfare Hub website?
- 3 . How useful did you find the Student Schemes/Scholarships section?
- 4 . Was the information provided about schemes and scholarships easy to understand?
- 5 . Did Student Welfare Hub save you time?
- 6 . Which feature did you find most useful?
- 7 . How satisfied are you with the overall experience?

#### 3. Collect responses from at least 50 people.

**Answer:-**

The dataset contains **55 responses**.

Therefore:

$$55 \geq 50$$

Hence, the requirement is satisfied.

#### 4. Store the collected data in a CSV file or Pandas DataFrame.

```
print("Data stored in Pandas DataFrame.")
print("Rows:", df.shape[0])
print("Columns:", df.shape[1])
```

... Data stored in Pandas DataFrame.  
Rows: 55  
Columns: 11

#### 5. Display the first five records of the collected data.

```
Timestamp  What is your current level of study? \
0  9/16/2026  Third Year
1  9/16/2026  Post Graduation
2  9/16/2026  Second Year
3  9/16/2026  Post Graduation
4  9/16/2026  Post Graduation

How easy was it to navigate and use the Student Welfare Hub website? \
0  Very Easy
1  Difficult
2  Easy
3  Neutral
4  Easy

How useful did you find the Student Schemes/Scholarships section? \
0  Usefull
1  Neutral
2  Usefull
3  Very usefull
4  Usefull
```

print(df.head())

#### 6. Identify the population and sample for the survey.

**Answer:-**

**Population:-**

All students/users who are potential users of the Student Welfare Hub.

**Sample:-**

The 55 students/respondents whose feedback is present in the collected dataset.

**Population** → All target Student Welfare Hub users

**Sample** → 55 respondents

### 7. Identify the sampling technique used to select the sample.

**Answer:-**

Sampling Technique:-

Voluntary Response / Convenience Sampling.

### 8. Identify the numerical and categorical variables in the collected data.

**Answer:-**

What is your current level of study?

How easy was it to navigate and use the Student Welfare Hub website?

How useful did you find the Student Schemes/Scholarships section?

Was the information provided about schemes and scholarships easy to understand?

Which feature of the Student Welfare Hub did you find most useful?

How satisfied are you with the overall experience of using the Student Welfare Hub?

How would you rate the accuracy and reliability of the information provided?

Would you recommend Student Welfare Hub to other students?

What problems or difficulties did you face while using the website?

### 9. Select one numerical variable for further analysis.

```
[18] accuracy_column = "How would you rate the accuracy and reliability of the information provided?"
accuracy = pd.to_numeric(
    df[accuracy_column],
    errors="coerce"
).dropna()
print("Selected Numerical Variable:")
print(accuracy_column)
print("\nNumber of valid observations:", len(accuracy))

... Selected Numerical Variable:
How would you rate the accuracy and reliability of the information provided?

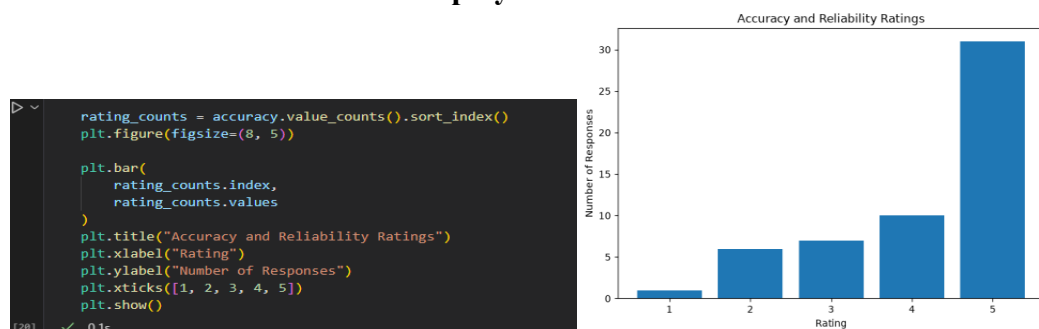
Number of valid observations: 55
```

### 10. Calculate the sample size, mean, median, and standard deviation of the selected variable.

```
[48] sample_size = accuracy.size
mean_value = accuracy.mean()
median_value = accuracy.median()
std_value = accuracy.std()
print("Sample Size:", sample_size)
print("Mean:", round(mean_value, 4))
print("Median:", median_value)
print("Standard Deviation:", round(std_value, 4))

... Sample Size: 55
Mean: 4.1636
Median: 5.0
Standard Deviation: 1.1347
```

### 11. Create a suitable chart to display the collected data.



### 12. Write two observations from the survey data.

```
[21] rating_counts = accuracy.value_counts().sort_index()
print("Rating Distribution:")
print(rating_counts)
highest_rating_count = rating_counts.get(5, 0)
high_rating_count = rating_counts.get(4, 0) + rating_counts.get(5, 0)
print("\nObservation 1:")
print(
    f"{highest_rating_count} respondents gave a rating of 5."
)
print("\nObservation 2:")
print(
    f"{high_rating_count} respondents gave a rating of 4 or 5."
)
```

```
... Rating Distribution:
How would you rate the accuracy and reliability of the information provided?
1    1
2    6
3    7
4   10
5   31
Name: count, dtype: int64

Observation 1:
31 respondents gave a rating of 5.

Observation 2:
41 respondents gave a rating of 4 or 5.
```

## Part B: Sampling Distribution and Central Limit Theorem

### 13. Select a numerical variable from the collected data.

```
selected_variable = accuracy
print("Selected Variable:")
print("Accuracy and Reliability Rating")

print("\nNumber of observations:", len(selected_variable))
```

```
... Selected Variable:
Accuracy and Reliability Rating

Number of observations: 55
```

### 14. Select a sample of 30 observations randomly from the data.

```
random_sample = accuracy.sample(
    n=30,
    random_state=42
)

print("Random Sample of 30 Observations:")
print(random_sample.tolist())
```

```
... Random Sample of 30 Observations:
[4, 3, 3, 3, 5, 5, 5, 4, 5, 5, 2, 5, 5, 4, 5, 5, 4, 5, 3, 4, 5, 5,
```

### 15. Calculate the mean of the selected sample.

```
sample_mean = random_sample.mean()

print("Mean of Random Sample:")
print(round(sample_mean, 4))
```

```
... Mean of Random Sample:
4.3
```

### 16. Repeat the random sampling process 100 times.

```
sample_size = 30
number_of_samples = 100
sample_means = []
for i in range(number_of_samples):
    sample = accuracy.sample(
        n=sample_size
    )
    mean = sample.mean()
    sample_means.append(mean)
print("Number of random samples:", len(sample_means))
```

```
... Number of random samples: 100
```

### 17. Calculate and store the mean of each sample.

```
sample_means = []
for i in range(100):
    sample = accuracy.sample(
        n=30
    )
    sample_mean = sample.mean()
    sample_means.append(sample_mean)
sample_means = np.array(sample_means)
print("Sample means stored successfully.")
print("Total sample means:", len(sample_means))
```

```
... Sample means stored successfully.
Total sample means: 100
```

### 18. Display the first 10 sample means.

```
print("First 10 Sample Means:")
for i, mean in enumerate(sample_means[:10], 1):
    print(f"Sample {i}: {mean:.4f}")
```

```
... First 10 Sample Means:
Sample 1: 4.1667
Sample 2: 4.3000
Sample 3: 4.0667
Sample 4: 4.0333
Sample 5: 4.0667
Sample 6: 4.1000
Sample 7: 3.8667
Sample 8: 4.2667
Sample 9: 4.4667
Sample 10: 4.1667
```

### 19. Calculate the mean of all the sample means.

```
mean_of_sample_means = sample_means.mean()
print(
    "Mean of all sample means:",
    round(mean_of_sample_means, 4)
)
```

```
... Mean of all sample means: 4.158
```

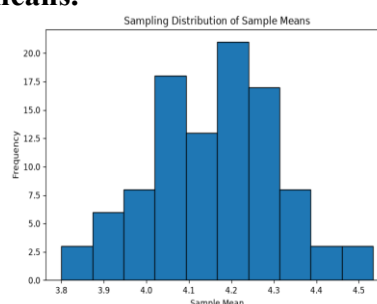
### 20. Calculate the standard deviation of the sample means.

```
std_sample_means = sample_means.std(ddof=1)
print(
    "Standard Deviation of Sample Means:",
    round(std_sample_means, 4)
)
```

```
... Standard Deviation of Sample Means: 0.1517
```

### 21. Create a histogram of the sample means.

```
plt.figure(figsize=(8, 5))
plt.hist(
    sample_means,
    bins=10,
    edgecolor="black"
)
plt.title("Sampling Distribution of Sample Means")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.show()
```



## 22. Observe the shape of the sampling distribution.

```
print("Mean of Sampling Distribution:",
      round(sample_means.mean(), 4))
print("Standard Deviation:",
      round(sample_means.std(ddof=1), 4))
print("\nObservation:")
print(
    "The sampling distribution is approximately "
    "bell-shaped and concentrated around its mean."
)
```

```
... Mean of Sampling Distribution: 4.158
Standard Deviation: 0.1517

Observation:
The sampling distribution is approximately bell-shaped and concentrated around its mean.
```

## 23. Compare the original sample mean with the mean of the sampling distribution.

```
original_mean = accuracy.mean()
sampling_distribution_mean = sample_means.mean()
difference = abs(
    original_mean - sampling_distribution_mean
)
print("Original Dataset Mean:",
      round(original_mean, 4))
print("Mean of Sampling Distribution:",
      round(sampling_distribution_mean, 4))
print("Absolute Difference:",
      round(difference, 4))
```

```
... Original Dataset Mean: 4.1636
Mean of Sampling Distribution: 4.158
Absolute Difference: 0.0056
```

## 24. Compare the sampling distributions obtained for different sample sizes.

```
sample_sizes = [10, 20, 30, 40]
distributions = {}
rng = np.random.default_rng(42)
for size in sample_sizes:
    means = []
    for i in range(100):
        random_state = int(
            rng.integers(0, 1000000)
        )
        sample = accuracy.sample(
            n=size,
            random_state=random_state
        )
        means.append(sample.mean())
    distributions[size] = np.array(means)
```

```
for size in sample_sizes:
    means = distributions[size]
    print(
        f"Sample Size = {size}"
    )
    print(
        "Mean:",
        round(means.mean(), 4)
    )
    print(
        "Standard Deviation:",
        round(means.std(ddof=1), 4)
    )
    print()
```

```
... Sample Size = 10
Mean: 4.188
Standard Deviation: 0.3285

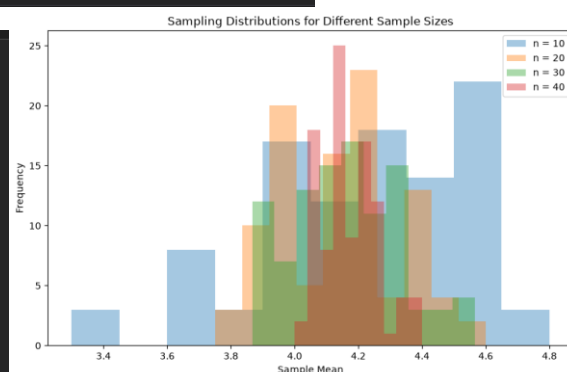
Sample Size = 20
Mean: 4.142
Standard Deviation: 0.1883

Sample Size = 30
Mean: 4.1613
Standard Deviation: 0.1595

Sample Size = 40
Mean: 4.1695
Standard Deviation: 0.0876
```

```
plt.figure(figsize=(10, 6))
for size in sample_sizes:
    plt.hist(
        distributions[size],
        bins=10,
        alpha=0.4,
        label=f"n = {size}"
    )
plt.title("Sampling Distributions for Different Sample Sizes")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.legend()

plt.show()
```



## 25. Calculate the standard error of the sample mean.

```
n = 30
sample_standard_deviation = accuracy.std(ddof=1)
standard_error = (
    sample_standard_deviation /
    np.sqrt(n)
)
print("Sample Standard Deviation:",
      round(sample_standard_deviation, 4))
print("Sample Size:", n)
print("Standard Error:",
      round(standard_error, 4))
```

```
... Sample Standard Deviation: 1.1347
Sample Size: 30
Standard Error: 0.2072
```

## 26. Explain how the Central Limit Theorem is demonstrated through the experiment.

Answer:-

### CENTRAL LIMIT THEOREM OBSERVATION

The original dataset contains individual ratings from 1 to 5.

We repeatedly selected random samples of size 30 and calculated their means.

The distribution of these sample means became approximately bell-shaped.

The mean of the sampling distribution was close to the original dataset mean.

Therefore, the experiment demonstrates the Central Limit Theorem.

**27. Use the sample mean to estimate the population mean.**

```
estimated_population_mean = sample_means.mean()
print(
    "Estimated Population Mean:",
    round(estimated_population_mean, 4)
)
print(
    "Original Dataset Mean:",
    round(accuracy.mean(), 4)
)
```

```
... Estimated Population Mean: 4.158
Original Dataset Mean: 4.1636
```

**28. Write a conclusion based on the sampling distribution**

**Answer:-**

**CONCLUSION:-**

The Student Welfare Hub feedback dataset contains 55 responses.

The mean accuracy and reliability rating was 4.1636.

By repeatedly selecting samples of 30 observations and calculating their means, a sampling distribution was obtained.

The mean of the sampling distribution was 4.1580.

The sampling distribution was approximately bell-shaped and became narrower as the sample size increased.

This experiment demonstrates the Central Limit Theorem and shows that the sample mean can be used to estimate the population mean.